

# Les classes en Python

---

# Introduction

```
>>> class Orange:
    pass

>>> Orange
<class '__main__.Orange'>

>>> type(Orange)
<class 'type'>

>>> orange1 = Orange()
>>> orange1
<__main__.Orange object at 0x00000205ABCF3F70>

>>> isinstance(orange1, Orange)
True

>>> dir(orange1)
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__']

>>> orange1.couleur = "orange"
>>> dir(orange1)
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'couleur']

>>> orange1.couleur
'orange'

>>> |
```

**dir()** : retourne les propriétés et méthodes d'un objet

**\_\_dict\_\_** : les attributs et valeurs d'un objet créés dynamiquement

```
>>> orange1.__dict__ # attributs créés dynamiquement
{'couleur': 'orange'}

>>> orange1.gout = "délicieux"
>>> orange1.__dict__
{'couleur': 'orange', 'gout': 'délicieux'}

>>> orange2 = Orange()
>>> orange2.__dict__
{}

>>> del orange1.couleur
>>> orange1.__dict__
{'gout': 'délicieux'}

>>> |
```

# Scriptage (Python Tutor)

## Python Tutor: Visualize code in Python, JavaScript, C, C++, and Java

Python 3.6  
([known limitations](#))

```
1 # classes.py
2 class Orange:
3     forme = "rond" # attribut de classe, constant por
4     def __init__(self, couleur="orange", taille="stan
5         self.couleur = couleur # attribut d'instance
6         self.taille = taille # attribut d'instance
7         self.masse = masse # attribut d'instance
8     def augmente_masse(self, valeur):
9         self.masse += valeur # instance
10    def change_forme(self, f):
11        Orange.forme = f # classe
12    if __name__ == "__main__":
13        orange1 = Orange()
14        orange2 = Orange()
15        print("Attributs de classe :", orange1.forme, ora
16        print("Attributs d'instance :", orange1.masse, or
17        orange1.change_forme("oval")
18        orange1.augmente_masse(100)
19        print("Attributs de classe :", orange1.forme, ora
20        print("Attributs d'instance :", orange1.masse, or
```

[Edit this code](#)

→ line that just executed

→ next line to execute

<< First < Prev Next > Last >>

Done running (26 steps)

[Customize visualization](#)

Print output (drag lower right corner to resize)

```
Attributs de classe : rond rond
Attributs d'instance : 0 0
Attributs de classe : oval oval
Attributs d'instance : 100 0
```

Frames

Global frame

Orange  
orange1  
orange2

Objects

Orange class

|                |                                                                                                                              |
|----------------|------------------------------------------------------------------------------------------------------------------------------|
| __init__       | function<br>__init__(self, couleur, taille, masse)<br>default arguments:<br>couleur "orange"<br>taille "standard"<br>masse 0 |
| augmente_masse | function<br>augmente_masse(self, valeur)                                                                                     |
| change_forme   | function<br>change_forme(self, f)                                                                                            |
| forme          | "oval"                                                                                                                       |

Orange instance

|         |            |
|---------|------------|
| couleur | "orange"   |
| masse   | 100        |
| taille  | "standard" |

Orange instance

|         |            |
|---------|------------|
| couleur | "orange"   |
| masse   | 0          |
| taille  | "standard" |

# Surcharge

```
# surcharge.py
class Orange:
    forme = "sphère" # attribut de classe, constant
    def __init__(self, couleur="orange", taille=10, masse=0):
        self.couleur = couleur # attribut d'instance
        self.taille = taille # attribut d'instance
        self.masse = masse # attribut d'instance (masse en gramme)
    def __add__(self, o): # surcharge de l'opérateur +
        return self.masse + o.masse
    def __len__(self): # surcharge de la fonction standard len
        return self.taille

if __name__ == "__main__":
    orange1 = Orange("orange", 10, 100)
    orange2 = Orange("orange", 20, 100)
    print(orange1 + orange2, len(orange1), len(orange2))
```

# Héritage

```
1 # heritage.py
2 class Fruit:
3     categorie = "alimentation"
4     def __init__(self, saveur=None, forme=None):
5         print("Je suis dans le constructeur de la classe Fruit")
6         self.saveur = saveur
7         self.forme = forme
8         print("Je viens de créer self.saveur et self.forme")
9     def montre_conseil(self, type_fruit, conseil):
10        print("Je suis dans la méthode .montre_conseil() de la "
11              "classe Fruit\n")
12        return (f"Instance {type_fruit}\n"
13              f"saveur: {self.saveur}, forme: {self.forme}\n"
14              f"conseil: {conseil}\n")
15 class Poire(Fruit):
16     def __init__(self, saveur=None, forme=None):
17         print("Je rentre dans le constructeur de Poire, et je vais appeler "
18               "le constructeur de la classe mère Fruit !")
19         Fruit.__init__(self, saveur, forme)
20         print("J'ai fini dans le constructeur de Poire, les attributs sont :\n"
21               f"self.saveur: {self.saveur}, self.forme: {self.forme}\n")
22     def __str__(self): # On surcharge str qui est utilisé par print()
23         print("Je rentre dans la méthode .__str__() de la classe Poire")
24         print("Je vais lancer la méthode .montre_conseil() héritée "
25               "de la classe Fruit")
26         return self.montre_conseil("Poire", "Bon en croustade !")
27 class Kiwi(Fruit):
28     def __init__(self, saveur=None, forme=None):
29         self.saveur = saveur # Je n'appelle pas le constructeur parent
30         self.forme = forme
31     def __str__(self):
32         return self.montre_conseil("Kiwi", "Bon en salade !")
33 if __name__ == "__main__":
34     # On crée une poire et un kiwi
35     poire = Poire(saveur="juteuse", forme="poire")
36     print("Poire>>", poire)
37     print("-----")
38     kiwi = Kiwi(saveur="douce", forme="ballon")
39     print("Kiwi>>", kiwi)
```

## Console ✕

```
>>> %Run heritage.py
```

```
Je rentre dans le constructeur de Poire, et je vais appeler le constructeur de la classe mère Fruit !
Je suis dans le constructeur de la classe Fruit
Je viens de créer self.saveur et self.forme
J'ai fini dans le constructeur de Poire, les attributs sont :
self.saveur: juteuse, self.forme: poire
```

```
Poire>> Je rentre dans la méthode .__str__() de la classe Poire
Je vais lancer la méthode .montre_conseil() héritée de la classe Fruit
Je suis dans la méthode .montre_conseil() de la classe Fruit
```

```
Instance Poire
saveur: juteuse, forme: poire
conseil: Bon en croustade !
```

```
-----
```

```
Kiwi>> Je suis dans la méthode .montre_conseil() de la classe Fruit
```

```
Instance Kiwi
saveur: douce, forme: ballon
conseil: Bon en salade !
```

# Encapsulation

```
1 # encapsulation.py
2 class Etudiant:
3     def __init__(self, nom, matricule):
4         # membre publique
5         self.nom = nom
6         # membre privé
7         self.__matricule = matricule
8
9     def affiche(self):
10         print("Détails de l'étudiant:", self.nom, self.__matricule)
11
12     # getter
13     def get_matricule(self):
14         return self.__matricule
15
16     # setter
17     def set_matricule(self, matricule):
18         self.__matricule = matricule
19
20 jessica = Etudiant('Jessica', 1234567)
21 jessica.affiche() # Détails de l'étudiant: Jessica 1234567
22 print(jessica.nom) # Jessica
23 #print(jessica.__matricule) # introuvable parce que privé
24 print(jessica.get_matricule()) # 1234567
25 jessica.nom = "Jessa"
26 jessica.__matricule = 7654321 # possible mais sans conséquence
27 jessica.affiche() # Détails de l'étudiant: Jessa 1234567
28 jessica.set_matricule(7654321)
29 jessica.affiche() # Détails de l'étudiant: Jessa 7654321
```

Console ✕

```
>>> %Run encapsulation.py
```

```
Détails de l'étudiant: Jessica 1234567
Jessica
1234567
```

```
Détails de l'étudiant: Jessa 1234567
Détails de l'étudiant: Jessa 7654321
```